

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Аналитический раздел	6
1.1 Анализ предметной области	6
1.2 Сравнительный анализ существующих решений	6
1.3 Диаграмма прецедентов разрабатываемого комплекса	7
1.4 Формализация и описание информации, подлежащей хранению разрабатываемым комплексом	8
1.5 Анализ решений для организации хранилища информации . . .	9
1.6 Анализ существующих баз данных на основе формализации данных	11
1.6.1 Дореляционные базы данных	11
1.6.2 Реляционные базы данных	11
1.6.3 Постреляционные базы данных	12
1.6.4 Вывод	12
1.7 Анализ существующих типов систем управления базами данных (СУБД) по способу доступа	12
1.7.1 Файл-серверные СУБД	12
1.7.2 Клиент-серверные СУБД	12
1.7.3 Встраиваемые СУБД	13
1.7.4 Сравнительный анализ рассмотренных СУБД	13
1.8 Выводы	14
2 Конструкторский раздел	15
2.1 Схема алгоритма монтирования устройства	15
2.2 Схема алгоритма удаления устройства из списка доверенных устройств	16
2.3 Схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения	16
2.4 Схема алгоритма проверки записи о монтировании на предмет актуальности	18
2.5 Описание сущностей проектируемой базы данных	19
2.6 Диаграмма проектируемой базы данных	20

2.7	Описание проектируемых ограничений целостности базы данных	21
3	Технологический раздел	23
3.1	Выбор языка и среды программирования	23
3.2	Выбор средства тестирования кода	23
3.3	Выбор встраиваемой системы управления реляционными базами данных (СУБД)	23
4	Исследовательский раздел	25
	ЗАКЛЮЧЕНИЕ	26
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	28
	ПРИЛОЖЕНИЕ А	29

ВВЕДЕНИЕ

Использование внешних запоминающих устройств создаёт риск попадания на компьютер вредоносного программного обеспечения и увеличивает вероятность несанкционированного копирования информации на сторонние носители.

Цель данной работы — разработка и реализация программно-алгоритмического комплекса, снижающего риски использования внешних запоминающих устройств.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать существующие решения;
- формализовать требования к разрабатываемому комплексу;
- разработать архитектуру программно-алгоритмического комплекса;
- обосновать выбор средств программной реализации;
- разработать программное обеспечение комплекса;
- провести тестирования комплекса;
- исследовать характеристики разработанного программного обеспечения.

1 Аналитический раздел

В данном разделе анализируется предметная область, существующие решения, формализуются требования к разрабатываемому комплексу.

1.1 Анализ предметной области

Наиболее часто встречающимся интерфейсом для подключения внешних запоминающих устройств к вычислительной технике является USB (Universal Serial Bus). К таким устройствам относятся флеш-карты и жёсткие диски. Наиболее простой способ устранения рисков, связанных с использованием внешних запоминающих устройств, — программное отключение USB-портов. Однако это не всегда может быть удобно, так как порты используются и устройствами ввода-вывода, например, мышью или клавиатурой. Более рациональным решением будет не полностью отключать порты, а программно изменять поведение компьютера при подключении устройства через USB-порт.

1.2 Сравнительный анализ существующих решений

Все существующие продукты делятся на две группы — корпоративные системы предотвращения утечек и утери данных, и решения для личного пользования. Корпоративные решения имеют распределённую архитектуру, рассчитаны на управление и мониторинг множества рабочих станций, и являются избыточными для персонального компьютера. В анализе участвуют только программы для личного использования.

Для анализа были выбраны следующие программные комплексы:

1. USBGuard [1];
2. USBFilter [2];
3. USBWall [3];
4. USBShield [4];
5. Ratool [5];
6. GiliSoft USB Lock [6];
7. USB Block [7].

В таблице 1.1 представлены результаты анализа существующих решений.

Таблица 1.1 – Результат анализа существующих решений

Критерий \ Номер решения							
	1	2	3	4	5	6	7
поддержка ОС (операционной системы) Linux Ubuntu	+	+	+	-	-	-	-
возможность создания списка доверенных устройств	+	+	+	+	-	+	+
возможность установки даты истечения периода доверия к устройству	-	-	-	-	-	-	-
наличие графического интерфейса	-	-	-	+	+	+	+
наличие обновлений и поддержки	+	-	-	+	+	+	+

Таким образом, представленные решения не соответствуют в полной мере поставленным критериям.

1.3 Диаграмма прецедентов разрабатываемого комплекса

На рисунке 1.1 представлена диаграмма прецедентов разрабатываемого комплекса.



Рисунок 1.1 – Диаграмма прецедентов комплекса

1.4 Формализация и описание информации, подлежащей хранению разрабатываемым комплексом

Формальное описание устройства может быть представлено следующими атрибутами:

1. идентификатор производителя;
2. идентификатор устройства;
3. уникальный серийный номер устройства;
4. наименование производителя;
5. наименование устройства.

Доверенное устройство характеризуется формальным описанием устройства и датой истечения доверия.

Запись о монтаже устройства описывается следующей информацией:

1. формальное описание устройства;
2. узел устройства;
3. путь монтирования;
4. режим монтирования (только для чтения или полный доступ).

1.5 Анализ решений для организации хранилища информации

Существует несколько основных способов организации хранилища, к которым относятся базы данных и плоские файлы.

В таблице 1.2 представлены результаты анализа хранилищ данных.

Таблица 1.2 – Результат анализа решений для хранения списка доверенных устройств

Критерий \ Решение	Плоские файлы	Базы данных
наличие встроенных механизмов поддержания целостности данных (ограничения, уникальность, проверки)	-	+
устойчивость хранилища к сбоям и некорректному завершению работы (восстановление согласованного состояния)	-	+
отсутствие дополнительных зависимостей у прикладного приложения	+	-
отсутствие возможности несанкционированного ручного просмотра и редактирования данных без специализированных инструментов	-	+
возможность выполнения выборки и фильтрации данных без дополнительной реализации алгоритмов поиска	-	+
отсутствие необходимости инициализации и настройки хранилища перед началом работы	+	-
возможность расширения структуры данных и добавления новых требований без переработки существующей логики хранения	-	+

На основе таблицы анализа для разрабатываемого комплекса предпочтительнее использовать базу данных.

1.6 Анализ существующих баз данных на основе формализации данных

Базы данных подразделяются на три основных типа по модели данных:

1. дореляционные;
2. реляционные;
3. постреляционные.

1.6.1 Дореляционные базы данных

Дореляционные базы данных [8] хранят записи в виде связей предок/потомок. Основными недостатками дореляционных баз данных являются избыточность данных и сложность выполнения запросов. Изменение схемы базы данных крайне ограничена. Целостность данных обеспечивается за счёт жёсткой структуры связей. Пример хранения записей в дореляционных базах представлен на рисунке 1.2.

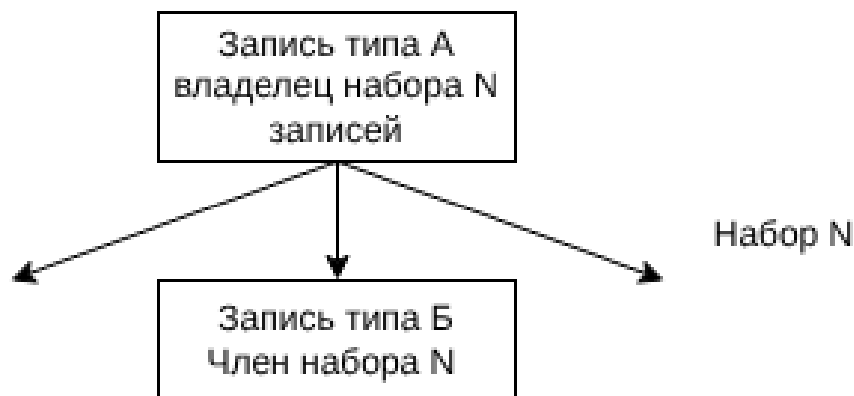


Рисунок 1.2 – Пример хранения записей в дореляционных базах данных

1.6.2 Реляционные базы данных

В реляционных базах данных данные хранятся в виде двумерных таблиц. Связь между ними обеспечивается общими столбцами, такими как идентификаторы. Табличная архитектура подходит для хранения структурированной информации. Целостность и непротиворечивость данных соблюдается при помощи ограничений.

1.6.3 Постреляционные базы данных

Постреляционные базы данных подходят для хранения неструктурированных данных, поддерживая логические связи между ними. Недостатком является сложность обеспечения целостности данных. Отличаются хорошей горизонтальной масштабируемостью, сложность запросов варьируется в зависимости от конкретной реализации.

1.6.4 Вывод

Для реализации поставленной задачи была выбрана реляционная модель, так как она лучше всего подходит для хранения структурированной информации, обеспечивая баланс между целостностью данных, простотой запросов и возможностями масштабирования.

1.7 Анализ существующих типов систем управления базами данных (СУБД) по способу доступа

СУБД по способу доступа к базе данных подразделяется на три основных вида [9]:

1. файл-серверные;
2. клиент-серверные;
3. встраиваемые.

1.7.1 Файл-серверные СУБД

При работе с файл-серверной СУБД обработка данных происходит на рабочем месте, а сервер применяется только как отдельный накопитель. Каждый пользователь сам использует информацию и вносит изменения в файлы данных и в индексные файлы.

1.7.2 Клиент-серверные СУБД

Клиент-серверная СУБД позволяет обмениваться клиенту и серверу минимально необходимыми объёмами информации. При этом основная вычислительная нагрузка ложится на сервер. Клиент может выполнять функции

предварительной обработки перед передачей информации серверу, но в основном его функции заключаются в организации доступа пользователя к серверу. В большинстве случаев клиент-серверная СУБД гораздо менее требовательна к пропускной способности компьютерной сети, чем файл-серверная СУБД, особенно при выполнении операции поиска в базе данных по заданным пользователем параметрам, т.к. для поиска нет необходимости получать на клиент весь массив данных: клиент передаёт параметры запроса серверу, а сервер производит поиск по полученному запросу в локальной базе данных. Результат выполнения запроса возвращается клиенту, который обеспечивает отображение результата пользователю. В файл изменения вносятся только после того, как сервер получит сообщение об успешной корректировке файла. Это исключает повреждение индексных файлов и существенно повышает быстродействие системы.

1.7.3 Встраиваемые СУБД

Встраиваемая система управления базой данных — это система, которая может быть связана с клиентским приложением таким образом, чтобы приложение и СУБД работали в едином адресном пространстве. Вместе со встроенной базой данных приложение может быть развернуто как единая программа. Благодаря связыванию приложения с базой данных, прикладная система выигрывает от снижения общей сложности и уменьшения затрат на администрирование.

1.7.4 Сравнительный анализ рассмотренных СУБД

В таблице 1.3 представлены результаты сравнения рассмотренных СУБД.

Таблица 1.3 – Результаты сравнения СУБД по способу доступа

Тип СУБД Критерий	Файл-серверная	Клиент-серверная	Встраиваемая
Отсутствие избыточности для локального однопользовательского приложения	+	-	+
Отсутствие необходимости сетевого взаимодействия	-	-	+
Отсутствие необходимости отдельного серверного процесса	-	-	+

Таким образом, для разрабатываемого программно-алгоритмического комплекса наиболее подходящей будет встраиваемая СУБД.

1.8 Выводы

В данном разделе была проанализирована предметная область, проведено сравнение существующих решений по сформулированным критериям. Программно-алгоритмический комплекс был формализован с использованием диаграммы прецедентов, описаны представления ключевых структур данных, проанализированы способы организации хранилища, выбраны наиболее подходящие типы базы данных и СУБД.

2 Конструкторский раздел

В данном разделе приведены реализуемые схемы алгоритмов, описаны сущности и ограничения целостности проектируемой базы данных.

2.1 Схема алгоритма монтирования устройства

На рисунке 2.1 представлена схема алгоритма монтирования устройства при его подключении к компьютеру.

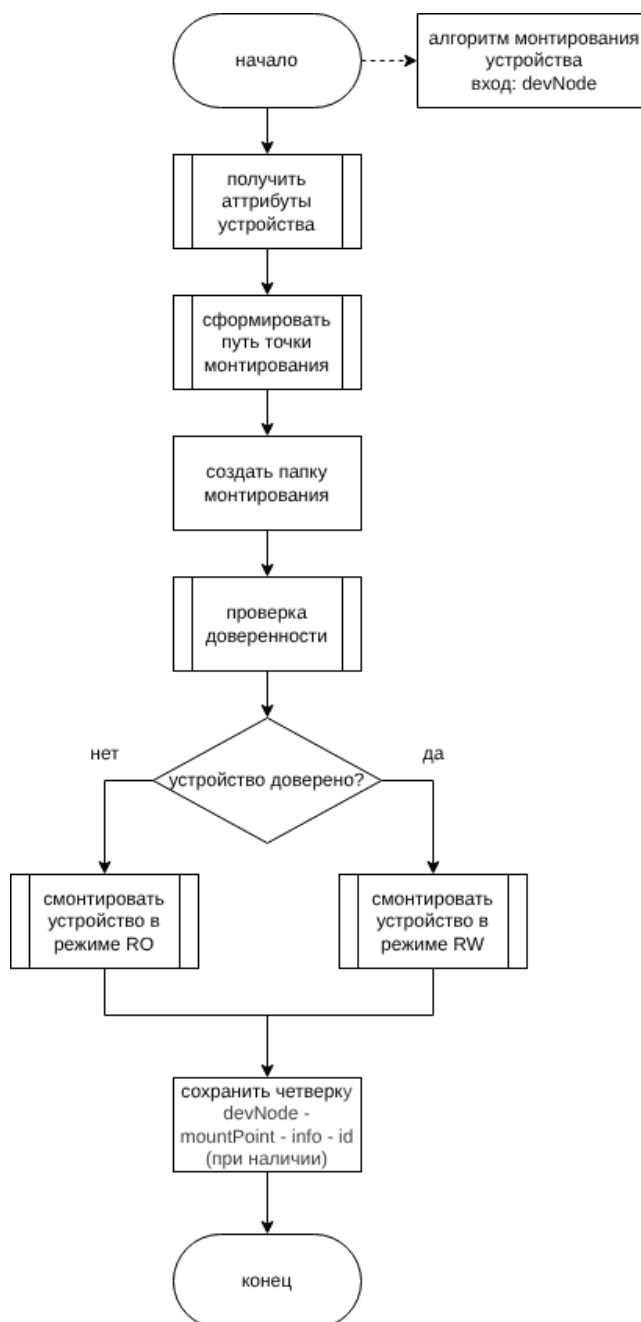


Рисунок 2.1 – Схема алгоритма монтирования

2.2 Схема алгоритма удаления устройства из списка доверенных устройств

На рисунке 2.2 представлена схема алгоритма удаления устройства из списка доверенных устройств.

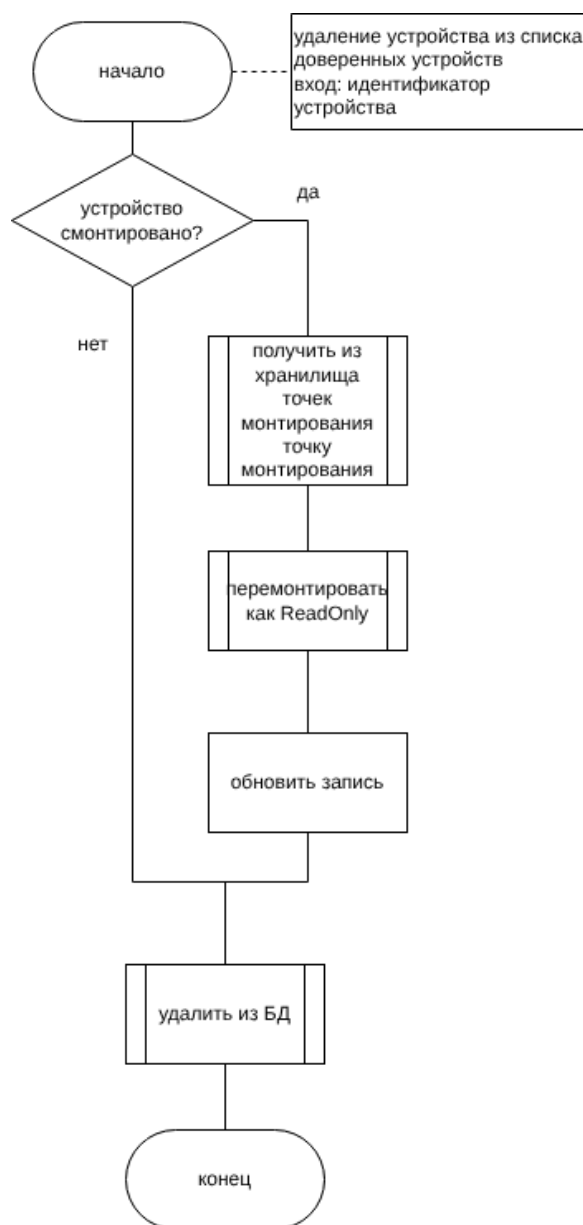


Рисунок 2.2 – Схема алгоритма удаления устройства

2.3 Схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения

На рисунке 2.3 представлена схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения.

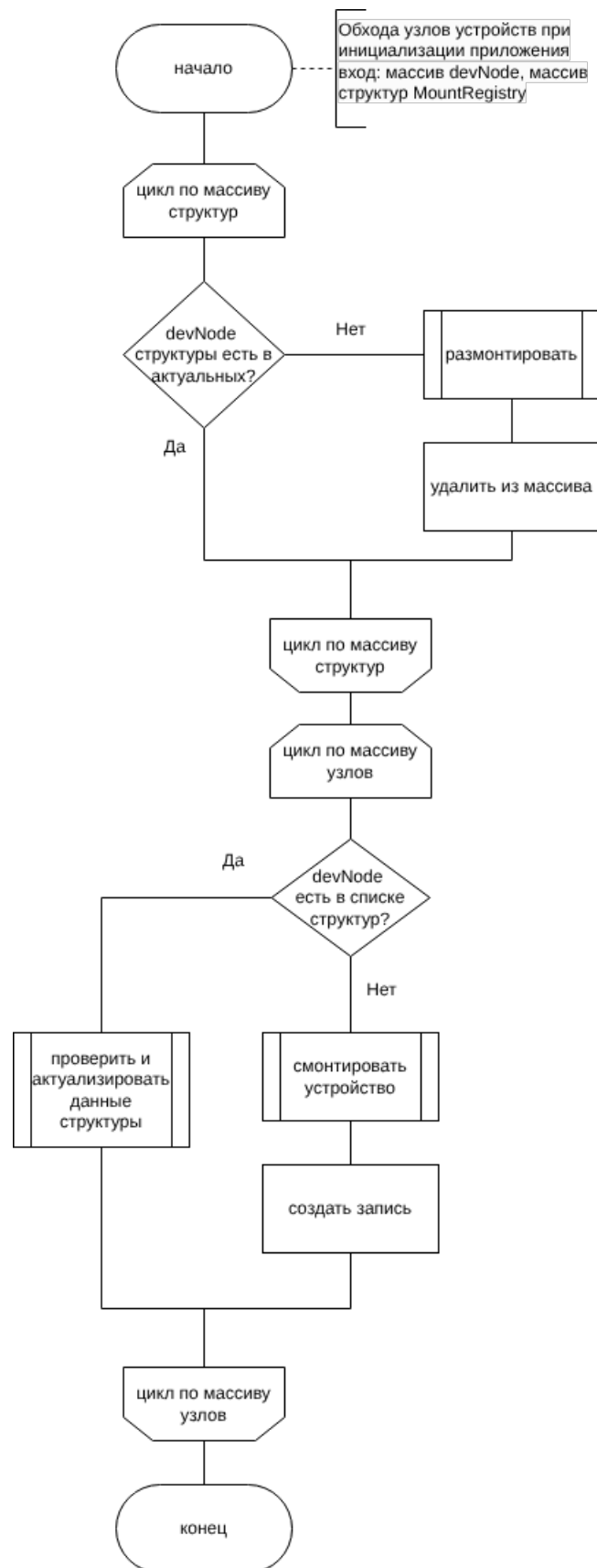


Рисунок 2.3 – Схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения

2.4 Схема алгоритма проверки записи о монтировании на предмет актуальности

На рисунке 2.4 представлена схема алгоритма проверки записи о монта-
ровании на предмет актуальности.

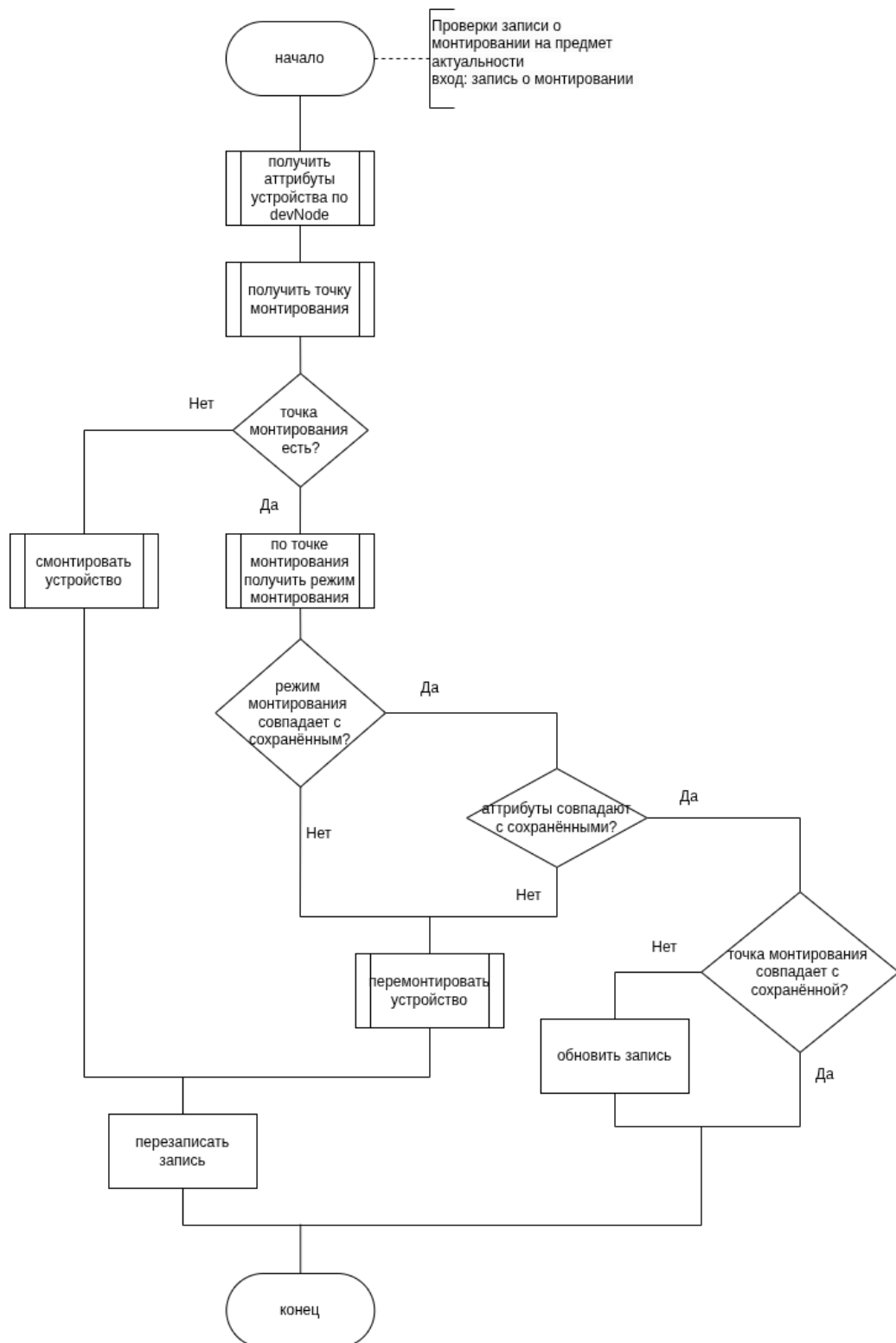


Рисунок 2.4 – Схема алгоритма проверки записи о монтировании на предмет актуальности

2.5 Описание сущностей проектируемой базы данных

База данных содержит следующие таблицы:

- DeviceInfo — описание устройства:
 1. id — первичный ключ;
 2. vendorId — идентификатор производителя;
 3. productId — идентификатор продукта;
 4. serial — серийный номер;
 5. productName — название производителя;
 6. vendorName — название устройства.
- Device — описание записи в списке доверенных устройств:
 1. id — первичный ключ;
 2. deviceInfoId — идентификатор описания устройства;
 3. validTo — дата окончания периода доверия.
- MountRecord — описание события монтирования:
 1. id — первичный ключ;
 2. deviceInfoId — идентификатор описания устройства;
 3. devNode — узел устройства;
 4. mountPoint — точка монтирования;
 5. mode — режим монтирования.

2.6 Диаграмма проектируемой базы данных

На рисунке 2.5 представлена диаграмма проектируемой базы данных.

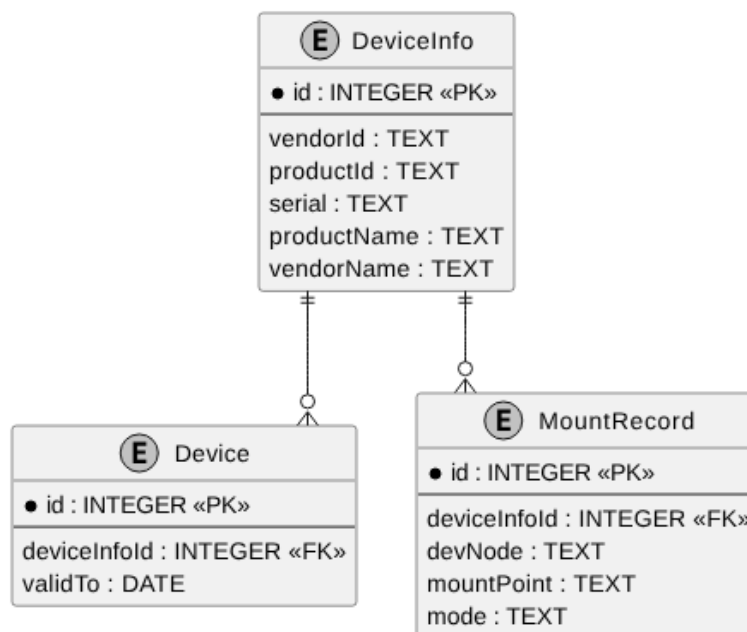


Рисунок 2.5 – Диаграмма базы данных

2.7 Описание проектируемых ограничений целостности базы данных

На значения полей базы данных накладываются следующие ограничения целостности:

— DeviceInfo:

1. поля `vendorId` и `productId` должны содержать только цифры или строчные или заглавные латинские буквы, их длина должна равняться 4-ём символам, они не могут быть пустыми;
2. поле `serial` не может быть пустым;
3. комбинация полей `vendorId`, `productId`, `serial` должна быть уникальной.

— MountRecord:

1. поле `deviceInfoId` является внешним ключом, ссылающимся на таблицу `DeviceInfo`, должно быть непустым и уникальным;
2. `devNode` должно быть непустым и уникальным;
3. `mountPoint` должно быть непустым и уникальным;

4. mode должно быть непустым и принимать только значения «RO» и «RW».
- Device: поле deviceInfoId является внешним ключом, ссылающимся на таблицу DeviceInfo, должно быть непустым и уникальным, а validTo должно содержать дату в формате «ГГГГ-ММ-ДД».

3 Технологический раздел

3.1 Выбор языка и среды программирования

Для разработки серверной части программно-алгоритмического комплекса был выбран язык программирования C++ [10], что обусловлено следующими факторами:

1. встроенные возможности объектно-ориентированного программирования, позволяющие создавать легко расширяемые приложения;
2. совместимость с языком C, возможность прямого вызова функций монтирования устройств;
3. поддержка многопоточности, необходимой для параллельной обработки событий;
4. стандартная библиотека предоставляет множество контейнеров, упрощающих разработку.

В качестве среды разработки был выбран Visual Studio Code [11] — кроссплатформенная среда, функциональность которой может быть увеличена с использованием дополнительных расширений.

3.2 Выбор средства тестирования кода

Для тестирования был выбран фреймворк GoogleTests [12]. Данный продукт позволяет создавать unit-тесты, e2e-тесты и интеграционные тесты. Разработан специально для C++, и использует стандартный его синтаксис. Позволяет реализовывать Mock-объекты для изоляции зависимостей при тестировании отдельных компонентов системы. Также фреймворк поддерживает параметризованные тесты, что позволяет проверять поведение методов на различных наборах входных данных без дублирования кода тестов.

3.3 Выбор встраиваемой системы управления реляционными базами данных (СУБД)

Для сравнения были выбраны следующие известные системы:

1. SQLite [13];

2. DuckDB [14];
3. H2 Database [15];
4. Apache Derby [16].

Таблица 3.1 – Результат анализа существующих решений

Критерий	Номер решения			
	1	2	3	4
Поддержка Linux	+	+	+	+
Поддержка C++	+	+	-	-
Бесплатное распространение	+	+	+	+
Поддержка внешних ключей	+	+	+	+
Поддержка ограничений	+	+	+	+
Наличие обновлений и поддержки	+	+	+	-
Отсутствие внешних зависимостей	+	+	-	-
Размер библиотеки(Мегабайты)	1.5	70	2.5	6.1

На основе анализа, для разрабатываемого комплекса больше всего подходит SQLite.

4 Исследовательский раздел

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. USBGuard; [Электронный ресурс]. — Режим доступа: <https://usbguard.github.io/> (дата обращения: 01.03.2026).
2. Making USB Great Again with USBFILTER — a USB layer firewall in the Linux kernel; [Электронный ресурс]. — Режим доступа: <https://davejingtian.org/2016/08/04/making-usb-great-again-with-usbfilter-a-usb-layer-firewall-in-the-linux-kernel/> (дата обращения: 01.03.2026).
3. USBWall Source Code; [Электронный ресурс]. — Режим доступа: <https://github.com/usbwall/usbwall?ysclid=mndivt3urh835120502> (дата обращения: 01.03.2026).
4. USBShield Source Code; [Электронный ресурс]. — Режим доступа: <https://github.com/USBShield/USBShield?ysclid=mndiyaq4ah326619085> (дата обращения: 01.03.2026).
5. Sordum — Ratool v1.4 (Removable Access tool); [Электронный ресурс]. — Режим доступа: <https://www.sordum.org/8104/ratool-v1-4-removable-access-tool/> (дата обращения: 01.03.2026).
6. Gilisoft USB Lock — Control USB ports, removable devices, and trusted-device access to reduce data leakage on Windows; [Электронный ресурс]. — Режим доступа: <https://www.gilisoft.com/product-usb-lock.htm> (дата обращения: 01.03.2026).
7. NewSoftwares — USBBlock. Restrict Access of Portable Drives; [Электронный ресурс]. — Режим доступа: <https://www.newsoftwares.net/usb-block/> (дата обращения: 01.03.2026).
8. Модели баз данных: учебное пособие / О.Е. Аврунев, В.М. Стасышин. — Новосибирск: Изд-во НГТУ, 2018. — 124 с.
9. Создание базы данных для разработки облачной платформы хранения и редактирования 3D моделей конфет / В. Г. Благовещенский, А. Е. Краснов, И. Г. Благовещенский [и др.] // Интеллектуальные автоматизированные управляющие системы в биотехнологических процессах : сборник докладов всероссийской научно-практической конференции,

Москва, 29 марта 2023 года. – Москва: РОССИЙСКИЙ БИОТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ; ЗАО «Университетская книга», 2023. – С. 85-96. – EDN NAJYYS.

10. Standard C++; [Электронный ресурс]. — Режим доступа: <https://isocpp.org/> (дата обращения: 01.03.2026).
11. Visual Studio Code — The open source AI code editor; [Электронный ресурс]. — Режим доступа: <https://code.visualstudio.com/> (дата обращения: 01.03.2026).
12. GoogleTest User's Guide; [Электронный ресурс]. — Режим доступа: <https://google.github.io/googletest/> (дата обращения: 01.03.2026).
13. SQLite Home Page; [Электронный ресурс]. — Режим доступа: <https://www.sqlite.org/> (дата обращения: 01.03.2026).
14. DuckDB — An in-process SQL OLAP database management system; [Электронный ресурс]. — Режим доступа: <https://duckdb.org/> (дата обращения: 01.03.2026).
15. H2 Database Engine — Welcome to H2, the Java SQL database; [Электронный ресурс]. — Режим доступа: <https://h2database.com/html/main.html> (дата обращения: 01.03.2026).
16. Apache Derby; [Электронный ресурс]. — Режим доступа: <https://db.apache.org/derby/> (дата обращения: 01.03.2026).

ПРИЛОЖЕНИЕ А